

14_7_maggio

Cache della memoria vs. Memoria virtuale

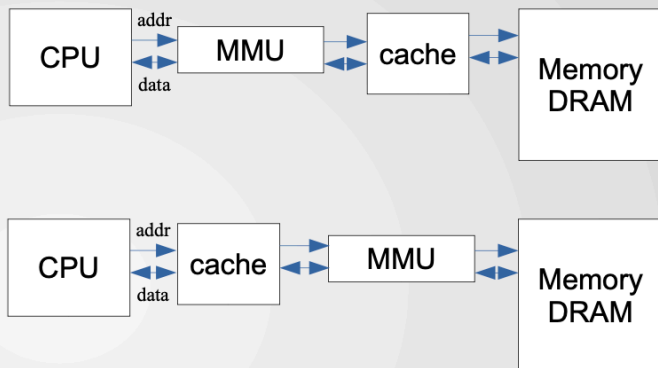
- La **cache della memoria** può essere:

- **basata sugli indirizzi fisici:**

- non serve invalidarla sul context-switch;
- caching poco efficace;

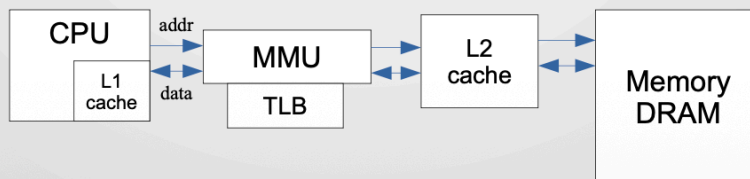
- **basata sugli indirizzi virtuali:**

- servono gli ASID per non invalidarla ogni volta;
 - scala male su L2;
- maggiore efficacia.



- Cosa si usa **in pratica?**

- cache L1 basata su indirizzi virtuali;
- cache L2 e successive basate su indirizzi fisici.



Cache: Prima o Dopo la MMU?

Il posizionamento della cache rispetto alla MMU determina se il sistema lavora con indirizzi fisici o virtuali, influenzando le prestazioni e la gestione del context-switch.

1. Cache basata su indirizzi fisici (Posizionata dopo la MMU)

In questa configurazione, la cache si trova tra la MMU e la RAM.

- **Funzionamento:** La CPU invia una richiesta alla MMU, che effettua la traduzione da indirizzo virtuale a fisico; solo a questo punto la richiesta arriva alla cache.
- **Vantaggi:** Non è necessario azzerare (invalidare) la cache durante un context-switch. Poiché gli indirizzi fisici sono univoci per l'intero sistema, i dati del processo P1 e del processo P2 possono coesistere nella cache senza ambiguità.
- **Svantaggi:** La traduzione dell'indirizzo da parte della MMU può diventare un collo di bottiglia, rendendo l'accesso ai dati più lento (caching meno efficace in termini di velocità pura).

2. Cache basata su indirizzi virtuali (Posizionata tra CPU e MMU)

In questa configurazione, la cache viene consultata direttamente utilizzando l'indirizzo virtuale generato dalla CPU.

- **Vantaggi:** Maggiore efficacia e velocità, poiché si evita il ritardo della traduzione della MMU in caso di *cache hit*.
- **Svantaggi :** Se non azzerassimo la cache al cambio di processo, P2 potrebbe accedere erroneamente ai dati di P1 (poiché entrambi usano gli stessi indirizzi virtuali locali).
- **Soluzione:** Per evitare l'azzeramento continuo, si utilizzano gli **ASID** (Address Space Identifier). Etichettando ogni linea di cache con l'identificativo del processo, l'hardware può distinguere tra indirizzi virtuali identici appartenenti a processi diversi. Tuttavia, questo sistema ha un costo hardware che aumenta proporzionalmente alla dimensione della cache.

3. Implementazione pratica.

Nella realtà, si adotta una combinazione delle due strategie per massimizzare le prestazioni:

- **Cache L1:** Solitamente interna alla CPU e basata su **indirizzi virtuali** per garantire la massima velocità.
- **Cache L2 (e successive):** Progettate per lavorare con **indirizzi fisici** (dietro la MMU). Sono più capienti e leggermente più lente della L1, ma non richiedono l'azzeramento al context-switch.

Il lavoro in parallelo:

Per ottimizzare i tempi, il sistema non attende l'esito di una componente prima di attivare l'altra:

1. Quando la CPU genera un indirizzo virtuale, inoltra contemporaneamente la richiesta alla **Cache L1** (per un riscontro immediato) e alla **MMU** (per avviare la traduzione). Se la L1 riporta un *cache miss*, la pratica di traduzione per accedere ai livelli successivi è già in corso.
2. **Scrematura tramite Offset:** Mentre la MMU traduce l'indirizzo virtuale in fisico per consultare la **Cache L2**, è possibile iniziare a lavorare. Poiché l'ultima parte dell'indirizzo (l'**offset**) è identica sia nell'indirizzo virtuale che in quello fisico, il sistema può usarla immediatamente per identificare il set di linee potenziali nella cache L2, riducendo i tempi di attesa dell'output finale della MMU.

Algoritmi di sostituzione delle pagine

- In caso di **page fault** ed in assenza di frame liberi è necessario scegliere una **pagina vittima**;
 - **come sceglierla?**
 - problema simile alla **gestione delle cache**;
 - **obiettivo**: minimizzare il numero di page fault in futuro;
- **soluzione ottimale** (algoritmo **OPT**):
 - scegliamo la pagina che verrà referenziata in un futuro più lontano;
 - ottimale ma **difficilmente realizzabile**;
 - rappresenta comunque un **termine di paragone**.

Algoritmi di Sostituzione delle Pagine

Quando un processo richiede una pagina non presente in RAM, si verifica un **page fault**. In questo caso, il SO deve intervenire per caricare la pagina mancante dalla memoria secondaria alla memoria principale e aggiornare la tabella delle pagine.

Pagina Vittima

Se la RAM è già piena e non ci sono frame liberi per ospitare la nuova pagina, il SO deve scegliere una **pagina vittima** da espellere per fare spazio.

- A differenza della cache (gestita dall'hardware), la gestione della memoria virtuale è affidata al **Sistema Operativo**.
- Ridurre al minimo il numero di page fault futuri, controllando i costi di overhead. La logica è simile a quella delle cache: decidere quale informazione rimuovere penalizzando il meno possibile l'esecuzione.

L'overhead è il lavoro che deve fare la CPU per pianificare/gestire la memoria senza quindi effettuare operazioni per i processi

L'Algoritmo Ottimale (OPT)

L'algoritmo **OPT** rappresenta la soluzione teorica perfetta per la sostituzione delle pagine.

- **funzionamento:** Tra tutte le pagine presenti in memoria, l'algoritmo sceglie come vittima quella che **verrà referenziata nel futuro più lontano**. Se sapessimo che una pagina non verrà consultata per molto tempo (o mai più), rimuoverla sarebbe la scelta ideale perché non genererebbe overhead immediato.
- **Vantaggi:** Garantisce il **costo minimo di overhead** (il minor numero possibile di page fault).
- **Limiti:** È un algoritmo **difficilmente realizzabile** nella pratica, poiché richiede al sistema operativo di prevedere il futuro comportamento del processo, informazione di cui il SO non dispone.
- Viene utilizzato principalmente come **termine di paragone**. Serve a valutare l'efficacia degli altri algoritmi reali confrontando le loro prestazioni con il "massimo teorico" offerto da OPT.

Algoritmo NRU

- Raccogliamo un po' di **statistiche** sull'uso della pagine caricate:
 - bit di **referenziamento** (R) e di **modifica** (M);
 - aggiornati tipicamente in **hardware**;
 - **azzerati** dal S.O.;
 - bit di referenziamento azzerato **periodicamente**;
- algoritmo **Not Recently Used** (NRU):
 - distinguiamo **4 classi di pagine**:
 - **classe 0**: non referenziato, non modificato;
 - **classe 1**: non referenziato, modificato;
 - **classe 2**: referenziato, non modificato;
 - **classe 3**: referenziato, modificato;
 - viene scelta una pagina dalla classe non vuota di numero più basso.

Algoritmo NRU (Not Recently Used)

L'algoritmo **NRU** è una soluzione pratica per la sostituzione delle pagine che si basa su statistiche d'uso minime raccolte tramite la tabella delle pagine.

I Bit di Stato

Per funzionare, l'algoritmo utilizza due bit specifici per ogni pagina, solitamente aggiornati direttamente dall'**hardware**:

- **Bit di Referenziamento (R):** viene impostato a **1** ogni volta che la pagina viene consultata (lettura o scrittura). Il Sistema Operativo provvede ad azzerarlo periodicamente per distinguere le pagine usate di recente da quelle vecchie.
- **Bit di Modifica (M):** viene impostato a **1** quando il contenuto della pagina viene scritto (pagina "dirty").

Classificazione delle Pagine

In caso di *page fault*, l'algoritmo suddivide le pagine presenti in RAM in **4 classi** basate sulla combinazione dei bit R ed M:

Classe	Stato Bit (R, M)	Descrizione
Classe 0	(0, 0)	Non referenziato, non modificato.
Classe 1	(0, 1)	Non referenziato, modificato.
Classe 2	(1, 0)	Referenziato, non modificato.
Classe 3	(1, 1)	Referenziato, modificato.

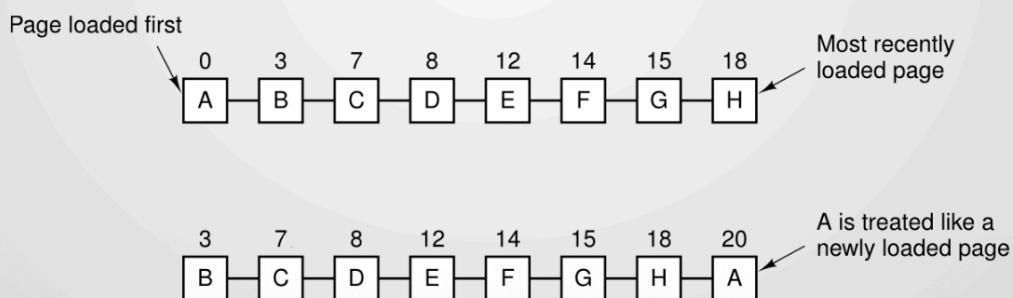
L'obiettivo è rimuovere una pagina che non sia stata usata recentemente.

1. Il Sistema Operativo analizza le classi in ordine crescente, dalla **0** alla **3**.
2. Viene scelta una pagina vittima appartenente alla **classe non vuota con il numero più basso**
 - *Es:* Se ci sono pagine nella Classe 0, se ne sceglie una casualmente tra quelle. Se la Classe 0 è vuota, si passa alla Classe 1, e così via.

Questo permette di eliminare prima le pagine che sono sia "vecchie" (non referenziate) che "pulite" (non modificate), minimizzando l'impatto sulle prestazioni del sistema.

Algoritmo FIFO e della seconda chance

- Algoritmo **First-In First-Out (FIFO)**:
 - viene rimossa la **pagina più vecchia**;
 - scelta non sempre felice: può rimuovere pagine, sì vecchie, ma magari molto usate;
- algoritmo della **Seconda Chance**:
 - si tiene conto dell'attuale stato del **bit R**;
 - viene rimossa la pagina più vecchia se non usata di recente.



Algoritmi FIFO e della Seconda Chance

Questi algoritmi si basano sul concetto di **età della pagina**, ovvero sul tempo trascorso da quando una pagina è stata caricata in RAM.

1. Algoritmo First-In First-Out (FIFO)

La logica è quella di una coda semplice: la prima pagina che entra è la prima a uscire.

- Viene rimossa sempre la pagina più vecchia (quella che si trova in "testa" alla coda).
- Basarsi solo sull'ordine di caricamento non è sempre una scelta felice. Una pagina potrebbe essere molto vecchia ma allo stesso tempo **molto usata** dal processo; rimuoverla causerebbe un immediato *page fault*, generando overhead inutile.

2. Algoritmo della Seconda Chance

Per migliorare il FIFO, questo algoritmo combina l'ordine temporale con le informazioni tracciate dalla MMU (il **bit di referenziamento R**). L'idea è quella di concedere una "seconda possibilità" alle pagine vecchie che sono state consultate di recente.

Come funziona:

1. Il sistema esamina la pagina più vecchia in testa alla coda.

2. Se il bit di referenziamento R è uguale a 1:

- La pagina non viene scartata.
- Il sistema **azzerà il bit R** (lo porta a 0) e sposta la pagina in **fondo alla coda**, trattandola come se fosse stata appena caricata. Questa è la sua "seconda chance".

3. Se il bit di referenziamento R è uguale a 0:

- La pagina viene scartata (vittima) poiché è sia vecchia che non utilizzata di recente.

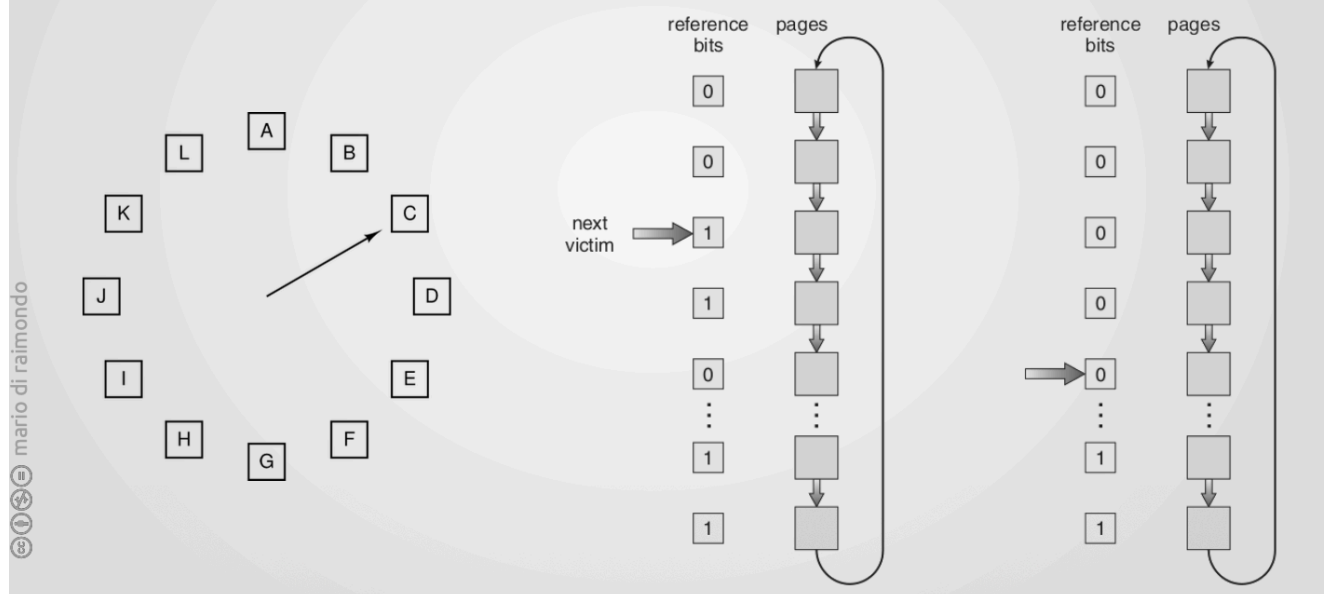
Esempio:

- La pagina **A** è la più vecchia (caricata al tempo 0).
- Invece di rimuoverla subito come farebbe il FIFO, il sistema controlla il suo bit R.
- Se R=1, **A** viene spostata alla fine della coda e il suo bit R viene azzerato. La ricerca della vittima prosegue quindi con la pagina **B**.

Questo aggiustamento permette di mantenere in RAM le pagine più importanti (frequentemente accedute) anche se sono state caricate molto tempo fa.

Algoritmo Clock

- L'idea dell'algoritmo della seconda chance è buona ma si può implementare in modo più efficiente:
 - algoritmo dell'orologio (**clock**).



Algoritmo dell'Orologio (Clock)

L'algoritmo **Clock** rappresenta un'evoluzione dell'algoritmo della Seconda Chance, progettata per essere implementata in modo più efficiente riducendo l'overhead dovuto

allo spostamento continuo delle pagine nella coda.

Struttura e Funzionamento

Invece di una coda lineare, le pagine vengono organizzate in una **coda circolare** (o lista circolare).

- **Il Puntatore:** Un puntatore (simile alla lancetta di un orologio) indica la prossima pagina da esaminare, che rappresenta la "testa" della coda.
- **Logica di Scelta:** Il criterio è identico a quello della Seconda Chance, basato sul **bit di referenziamento (R)**:
 - Quando si verifica un *page fault*, il sistema controlla la pagina indicata dal puntatore.
 - **Se il bit R è a 1:** la pagina riceve una "seconda possibilità". Il bit viene **posto a 0** e il puntatore avanza alla pagina successiva (seguendo il puntatore *next*) senza rimuoverla.
 - **Se il bit R è a 0:** la pagina viene identificata come **vittima** e scartata per fare spazio alla nuova pagina.

A differenza della Seconda Chance classica, non è necessario rimuovere fisicamente la pagina dalla testa per reinserirla in coda; è sufficiente spostare il puntatore e modificare il bit, rendendo l'operazione molto più rapida a livello hardware/software.

Esempio

Nell'immagine, il puntatore "next victim" punta a una pagina con bit R=1. Secondo la logica dell'algoritmo, quel bit viene azzerato (portato a 0) e il puntatore scorre verso il basso finché non incontra la prima pagina che ha già il bit R=0, che sarà effettivamente rimossa.

Algoritmo LRU

- Probabilmente le pagine più usate di recente lo saranno anche in futuro;
 - **idea:** rimuovere le pagine meno usate di recente;
- algoritmo **Least Recently Used (LRU)**:
 - buona idea ma non semplice e dispendiosa da implementare:
 - con supporto **hardware**:
 - **contatore** nella CPU e campi relativi nella tabella delle pagine;
 - via **software**:
 - algoritmo **NFU**
 - algoritmo **Aging**

Algoritmo Least Recently Used (LRU)

L'algoritmo **LRU** si basa sull'osservazione del comportamento passato del processo per cercare di prevedere le necessità future.

L'idea fondamentale è che le pagine usate più di recente hanno un'alta probabilità di essere usate ancora nel prossimo futuro. Al contrario, una pagina che non viene consultata da tempo è probabilmente meno utile e può essere rimossa senza causare overhead immediato.

Funzionamento

L'obiettivo è rimuovere la pagina che è stata utilizzata meno di recente. Sebbene sia un'ottima strategia teorica, la sua implementazione pratica è complessa:

- **Contatore:** Per tenere traccia dell'uso, è necessario aggiornare un contatore ogni volta che avviene un accesso alla pagina (lettura o scrittura).
- Una gestione precisa richiederebbe un contatore nella CPU e campi specifici all'interno della tabella delle pagine per memorizzare il valore del tempo di accesso.
- Data la difficoltà di implementazione hardware pura, si ricorre spesso ad algoritmi che approssimano il comportamento LRU via software, come:
 - **NFU (Not Frequently Used):** basato sulla frequenza d'uso.

- **Aging**: una tecnica più raffinata che simula il decadimento dell'importanza di una pagina nel tempo.

Mentre il FIFO guarda solo a quando la pagina è stata caricata, l'LRU cerca di fare la scelta migliore guardando a quando la pagina è stata effettivamente consultata l'ultima volta.

Algoritmo NFU

- LRU, anche se implementato in hardware, è dispendioso:
 - pensiamo ad una sua approssimazione;
- algoritmo **Not Frequently Used (NFU)**:
 - si tratta di una approssimazione (via software) dell'algoritmo LRU;
 - un **contatore** in ogni voce della tabella delle pagine;
 - periodicamente il valore del **bit R**, prima di essere azzerato, viene sommato a tale contatore;
 - viene rimossa la pagina con il **contatore più basso**;
- **problema**: può erroneamente privilegiare pagine che sono state molto utilizzate in passato ma che invece sono scarsamente usate di recente: queste lo saranno, probabilmente, anche nel prossimo futuro.

© mario di raimondo

Algoritmo NFU (Not Frequently Used)

Funzionamento del Contatore

L'idea centrale è associare un **contatore software** a ogni voce della tabella delle pagine.

- **Aggiornamento Periodico**: Il SO esamina periodicamente le pagine.
- **Somma del Bit R**: Prima che il bit di referenziamento (R) venga azzerato, il suo valore (0 o 1) viene sommato al contatore della rispettiva pagina.
- **Significato del valore**: Il contatore diventa quindi un conteggio cumulativo dei cicli di azzeramento in cui la pagina è risultata referenziata.

In caso di page fault, il SO cerca la pagina vittima basandosi sulla frequenza d'uso :

- Viene rimossa la pagina che presenta il **contatore più basso**.

- Si spera che un valore basso indichi una pagina utilizzata meno frequentemente nel tempo.

Problemi

Nonostante offra un criterio di scelta, guardare semplicemente al valore più basso può essere fuorviante:

- Il problema principale è che l'algoritmo può "privilegiare" erroneamente pagine che sono state utilizzate intensamente in un passato lontano, ma che ora non servono più.
- Queste pagine manterranno un contatore alto per molto tempo, occupando frame che dovrebbero essere assegnati a pagine usate meno in passato ma fondamentali nel presente (e probabilmente nel prossimo futuro).

Algoritmo di Aging

- **Algoritmo di Aging:**
 - ad ogni scadenza del clock:
 - **shift a destra** del contatore associato ad ogni pagina;
 - **accostamento a sinistra (come bit più significativo) del bit R.**

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00010000	10001000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000

Algoritmo di Aging

Invece di sommare semplicemente il bit di referenziamento, il sistema esegue un'operazione bit a bit più complessa a ogni scadenza del clock:

- **Shift a destra:** Il contatore di ogni pagina viene spostato di una posizione verso destra, scartando il bit meno significativo.

- **Inserimento del Bit R:** Il bit di referenziamento (R) attuale viene inserito nella posizione più a sinistra, diventando il **bit più significativo**.
- **Aggiornamento:** Questa operazione viene eseguita per ogni pagina sul relativo contatore software.

Il criterio di scelta rimane lo stesso: in caso di page fault, viene rimossa la pagina con il **contatore più piccolo**.

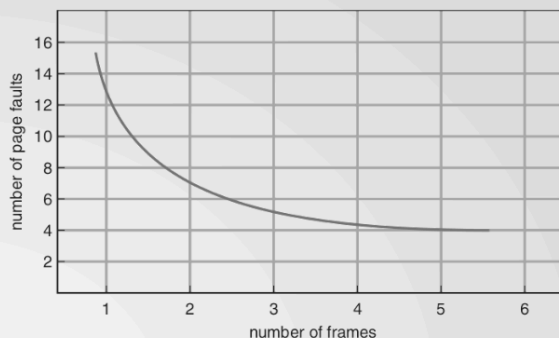
Scegliere il contatore più basso è coerente con la logica dell'algoritmo **NRU** (e LRU) per i seguenti motivi:

- Grazie allo shift a destra, il valore di un vecchio referenziamento diminuisce costantemente di peso a ogni ciclo di clock. Un bit R=1 registrato molti cicli fa varrà molto meno di un bit R=1 appena inserito nella posizione più significativa.
- Una pagina referenziata nell'ultimo ciclo avrà il bit più significativo a 1, risultando in un valore numerico molto alto. Una pagina non usata di recente avrà i bit più significativi a 0, portando a un valore numerico più piccolo.
- Scegliere il valore più piccolo significa identificare la pagina che non è stata referenziata per il maggior numero di cicli recenti, simulando così il comportamento "Not Recently Used" con una precisione più elevata.

In termini di costo, pur richiedendo operazioni di shift, l'Aging rimane una soluzione software efficiente che offre un'ottima approssimazione dell'algoritmo ottimale (OPT).

Confronto delle prestazioni

- Scegliamo una **metrica**:
 - numero di fault di pagina;
- a parità di condizioni, fissiamo:
 - ♦ **numero di frame:** 3 frame
 - ♦ **sequenza degli indirizzi virtuali a cui accedere;**
 - in modo equivalente: **sequenza compatta delle pagine;**
 - **7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1**
- **algoritmo OPT:**



7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2		2		2		2		2				7		
	0	0	0		0		4		0		0		0				0		
		1	1		3		3		3		1		1				1		

→ 9 fault di pagina

Prestazioni

Per confrontare l'efficacia dei diversi algoritmi di sostituzione, si utilizza come metrica principale il **numero di fault di pagina**.

Page Fault

Il grafico nell'immagine mostra il comportamento atteso del sistema:

- **All'aumentare del numero di frame** disponibili in RAM, il numero di page fault diminuisce progressivamente.
- Idealmente, con un numero di frame sufficiente a ospitare tutte le pagine richieste dal processo, il numero di fault tende a stabilizzarsi o ad azzerarsi (fatta eccezione per i fault iniziali necessari a caricare le pagine per la prima volta).

Configurazione del Test

Vengono fissate le seguenti condizioni:

- **Numero di frame:** 3 frame fisici disponibili.
- **Sequenza di accesso:** Una "sequenza compatta" di pagine virtuali: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1**.
 - Le ripetizioni consecutive vengono scartate nella sequenza compatta perché non genererebbero nuovi eventi rilevanti per la logica di sostituzione.

Applicazione dell'Algoritmo Ottimale (OPT)

L'algoritmo OPT effettua le scelte di sostituzione guardando al **futuro** anziché al passato.

1. Fase Iniziale (Riempimento):

- **Accesso a 7:** La RAM è vuota. Si verifica un page fault e la pagina 7 viene inserita nel primo frame.
- **Accesso a 0:** Page fault, la pagina 0 occupa il secondo frame.
- **Accesso a 1:** Page fault, la pagina 1 occupa il terzo frame. Ora la RAM è piena (7, 0, 1).

2. Sostituzione (Scelta della Vittima):

- **Accesso a 2:** Si verifica un page fault. Bisogna scegliere chi eliminare tra 7, 0 e 1.
- **OPT:** Guardando il futuro della sequenza, la pagina **7** è quella che verrà richiesta più avanti nel tempo rispetto a 0 e 1. Pertanto, la pagina 7 viene scartata e sostituita dalla pagina 2.
- Ad ogni fault, l'algoritmo analizza la sequenza futura per scartare la pagina il cui prossimo riferimento è il più lontano possibile.

In questo modo e con l'uso dell'algoritmo OPT, si ottengono complessivamente **9 fault di pagina**. Questo valore rappresenta il limite minimo (ottimale) con cui confrontare gli altri algoritmi reali.

Confronto delle prestazioni

- algoritmo **FIFO**:

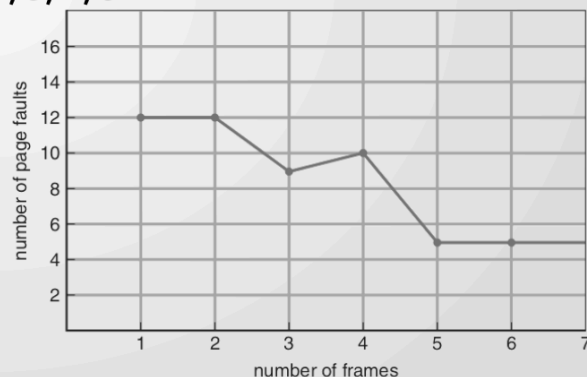
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2	2	4	4	4	0		0	0				7	7	7
	0	0	0		3	3	3	2	2	2		1	1				1	0	0
		1	1		1	0	0	0	3	3		3	2				2	2	1

→ 15 fault di pagina

- ma c'è qualcosa di strano...

- sequenza: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

- anomalia di Belady:** aumentando i frame disponibili, aumentano i fault di pagina (!?!)



Algoritmo FIFO e Anomalia di Belady

L'algoritmo **FIFO (First-In, First-Out)** è un metodo di sostituzione delle pagine che si basa sulla semplicità: la prima pagina che entra in memoria è la prima a essere scartata. Per valutarne le prestazioni, usiamo come indice il numero di **page fault** generati rispetto a una sequenza di accessi.

Il confronto con l'algoritmo OPT

Garantisce sempre il minor numero possibile di page fault. Tuttavia, FIFO presenta un comportamento che lo rende meno affidabile in certi scenari.

L'Anomalia di Belady

Ci si aspetta che all'aumentare della memoria RAM (ovvero dei frame disponibili), il numero di page fault diminuisca. FIFO soffre dell'**Anomalia di Belady**: un fenomeno per cui, aumentando i frame, il numero di fault di pagina può aumentare anziché scendere.

- 1 Frame:** si registrano **12 fault**.
- 2 Frame:** la situazione non cambia, restano **12 fault**.

- **3 Frame:** i fault scendono a **9**.
- **4 Frame:** **anomalia**; invece di scendere ancora, i fault **aumentano a 10**.

Il problema di fondo è che FIFO usa solo l'**età della pagina** come criterio per decidere cosa scartare. Questo non è sempre un parametro sensato: una pagina potrebbe essere in memoria da molto tempo (quindi "vecchia" per FIFO), ma essere usata spessissimo dal programma. Scartarla solo perché è arrivata per prima genera inutili page fault aggiuntivi.

Confronto delle prestazioni

- **algoritmo LRU:**

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1			
	0	0	0		0		0	0	3	3		3		0		0			
		1	1		3		3	2	2	2		2		2		7			

→ 12 fault di pagina

- non soffre dell'anomalia precedente, infatti vale la
 - **proprietà di inclusione:** l'insieme di pagine caricate avendo n frame è incluso in quello che si avrebbe avendo n+1 frame.
 - ♦ $B_t(n) \subseteq B_t(n+1) \quad \forall t, n$
- altri algoritmi visti:
 - **NFU, aging:** godono della proprietà di inclusione (appross. LRU);
 - **seconda chance, clock:** soffrono dell'anomalia (riducono a FIFO);
 - **NRU:** soffre dell'anomalia (riduce a FIFO).

L'Algoritmo LRU e la Proprietà di Inclusione

A differenza di FIFO, LRU si comporta molto bene anche su sequenze lunghe e **non soffre dell'Anomalia di Belady**.

La Proprietà di Inclusione. Il motivo per cui LRU è immune all'anomalia risiede nella **proprietà di inclusione**. Questa proprietà garantisce che, aumentando la memoria, le prestazioni non possano peggiorare.

L'insieme delle pagine caricate avendo n frame è incluso in quello che si avrebbe avendo n+1 frame. Se aggiungiamo uno spazio, questo ospiterà una pagina in più senza stravolgere l'ordine di quelle già presenti. LRU guarda a quando le pagine sono state referenziate l'ultima volta; avere più spazio permette solo di mantenere in RAM pagine che prima sarebbero state scartate, senza alterare il pattern di utilizzo.

Non tutti gli algoritmi godono di questa proprietà.

- **NFU e Aging:** Sono considerati approssimazioni di LRU e, per questo motivo, **godono della proprietà di inclusione**.
- **Second Chance e Clock:** Sebbene cerchino di migliorare la gestione della memoria, hanno un legame stretto con FIFO. In casi limite — ad esempio quando il bit di riferimento R è uguale per tutte le pagine (tutti 0 o tutti 1) — questi algoritmi **collassano su FIFO**, soffrendo dell'Anomalia di Belady.
- **NRU (Not Recently Used):** Anche questo algoritmo, basato sulla divisione delle pagine in 4 classi, soffre dello stesso problema. Se tutte le pagine finiscono nella stessa configurazione/classe, l'algoritmo si riduce a un semplice FIFO e può presentare l'anomalia.

Riepilogo sugli algoritmi

- **OPT:** non implementabile, ma utile come termine di paragone;
- **NRU*:** approssimazione rozza dell'LRU;
- **FIFO*:** può portare all'eliminazione di pagine importanti;
- **Seconda chance*:** un netto miglioramento rispetto a FIFO;
- **Clock*:** come S.C. ma più efficiente;
- **LRU:** eccellente idea (vicina a quella ottima) ma difficilmente realizzabile se non in hardware;
- **NFU:** approssimazione software abbastanza rozza dell'LRU;
- **Aging:** buona approssimazione di LRU con implementazione software efficiente.

* soffre dell'anomalia di Belady